ELSEVIER

Theoretical Computer Science 163 (1996) 211–238

Theoretical Computer Science

Exploiting few inversions when sorting: Sequential and parallel algorithms[1]

Christos Levcopoulos[a,*], Ola Petersson[b,2]

[a] *Department of Computer Science, Lund University, Box 118, S-221 00 Lund, Sweden*
[b] *Department of Mathematics, Statistics, and Computer Science, Växjö University, S-351 95 Växjö, Sweden*

Received September 1993; revised May 1995
Communicated by M. Crochemore

Abstract

We consider the problem of taking advantage of existing order within the input sequence when sorting. The measure of presortedness used is the number of inversions. Let X be a sequence of length n and let $Inv(X)$ be the (unknown) number of inversions in X. Our main results are:

- X can be sorted *in-place*, i.e. using only $O(\log n)$ bits of extra space, in time $O(n \log(Inv(X)/n))$, which is optimal with respect to the number of inversions.
- Given p processors on an EREW PRAM, X can be sorted in time

$$O\left(\frac{n \log(Inv(X)/n)}{p} + \log n\right),$$

which is optimal with respect to the number of inversions.

1. Introduction

It is well known that $\Omega(n \log n)$ time is necessary to sort n elements in both the worst case and the average case in a comparison-based model of computation [21]. Some instances are, however, easier and can be sorted faster. We identify this instance easiness with the amount of existing order (*presortedness*) in the sequence.

After being considered by, among others, Dijkstra [8], Guibas et al. [10], Knuth [15], and Mehlhorn [21], the concept of presortedness was eventually formalized by Mannila [20] in 1985, who studied different ways of measuring the presortedness of a sequence;

* Corresponding author. E-mail: christos@dna.lth.se.

[1] Preliminary versions of part of the results reported in this paper have been presented at 8th Conf. on Foundations of Software Technology and Theoretical Computer Science, 1988, and 8th Internat. Conf. on Fundamentals of Computation Theory, 1991.

[2] E-mail: ola@dna.lth.se.

S0304-3975/96/$15.00 © 1996 — Elsevier Science B.V. All rights reserved
SSDI S0304-3975(95)00256-1

for instance, the number of ascending runs and the number of inversions. Mannila also studied the problem of how a sorting algorithm can take advantage of, and thereby adapt to, the existing order. A sorting algorithm is said to be *adaptive* with respect to a measure of presortedness if it sorts all sequences, but performs particularly well on those that have a high degree of presortedness. Note that most worst-case optimal sorting algorithms are not adaptive at all.

In this paper the measure of presortedness considered is the number of inversions, *Inv*, that is, the number of pairs of elements that are in wrong order. In Section 3 we present a simple generic algorithm, the *Block Routine*, from which sorting algorithms that adapt to *Inv* can be derived. Basically, the generic algorithm divides the input sequence into blocks of equal length and sorts each block by any worst-case optimal sorting algorithm. The blocks are then permuted such that their medians appear in sorted order. Now, elements causing the concatenation of consecutive pairs of blocks to be nonsorted are removed. The removed elements are then sorted separately, and finally merged with the concatenation of what remains in the blocks.

In Section 4 the Block Routine is applied in the construction of a sorting algorithm that optimally adapts to the number of inversions and uses only a constant amount, i.e., $O(\log n)$ bits, of extra space. Here, optimality means maximal adaptation in an asymptotic sense. It is also optimal with respect to another commonly used measure of presortedness, namely, *Rem*, the minimum number of elements needed to be removed from a sequence in order to leave a sorted sequence. The algorithm assumes a word length of $\Omega(\log n)$, and it relies on linear-time, in-place merging [13] as well as linear-time, in-place selection [16]. It further uses the technique of encoding pointers by swapping pairs of elements [22]. As all known algorithms that are optimal with respect to *Inv* require linear extra space this is a considerable improvement.

To adapt to *Inv* the Block Routine relies on linear-time selection, which is a bottleneck because of its implicit constant factors. We observe that if we knew the number of inversions prior to the sort then selection can be avoided. Such knowledge is not realistic in most applications; however, in Section 5 we show that an appropriate approximation of *Inv* can be obtained by counting the number of inversions in a random sample.

In Section 6 we attack the problem of how few inversions there must be in a sequence for it to be possible to sort it faster than a 'random' sequence. That is, how little existing order are we able to exploit?

In Section 7 we show how to implement the Block Routine efficiently in a parallel model of computation. The algorithm presented runs in logarithmic time and achieves linear speedup.

2. Preliminaries

Let $X = \langle x_1, \ldots, x_n \rangle$ be a *sequence* of n elements x_i from some totally ordered set. The *concatenation* of two sequences $X = \langle x_1, \ldots, x_n \rangle$ and $Y = \langle y_1, \ldots, y_m \rangle$ is the sequence $XY = \langle x_1, \ldots, x_n, y_1, \ldots, y_m \rangle$. A consecutive subsequence of X is called a

block of X. Further, let $|X|$ denote the *length* of X and $\|S\|$ the *cardinality* of a set S. Finally, let $\log x$ denote $\log_2(\max\{2,x\})$.

The measure of presortedness we consider is the most well studied; the number of *inversions* [10, 12, 15, 18, 20, 21]. For a sequence X of length n, the number of inversions in X is formally defined as

$$Inv(X) = \|\{(i,j) \mid 1 \leqslant i < j \leqslant n \text{ and } x_i > x_j\}\|.$$

From an operational point of view, $Inv(X)$ tells the minimum number of exchanges between adjacent elements needed to bring X into sorted order.

For the concept of optimality of a sorting algorithm with respect to a measure of presortedness, we refer the reader to Mannila [20] or Petersson and Moffat [23]. For *Inv*, Guibas et al. [10] proved

Lemma 1. *An algorithm that spends* $O(n \log(Inv(X)/n))$ *time to sort any sequence* X *of length* n *is optimal with respect to Inv.*

There are several sorting algorithms that match this bound, for instance Splitsort [18], Adaptive Heapsort [19], Local Insertion Sort [20], and A-Sort [21]. We note that all these algorithms use a linear amount of extra space. The only known in-place sorting algorithm that adapts to the number of inversions, Smoothsort [8], was shown not to be optimal by Hertel [12].

3. Generic algorithm

We introduce a simple generic algorithm for sorting a sequence X of length n. The algorithm makes use of an integer parameter, ℓ, to be specified later. For simplicity, we assume that ℓ divides n.

Block Routine(X,ℓ)

Step 1: *Sort blocks.* Divide X into n/ℓ blocks of length ℓ and sort each block. (If $\ell = n$, this is the only step executed.)

Step 2: *Sort medians.* Sort the sequence consisting of the medians of the blocks.

Step 3: *Permute blocks.* Permute the blocks such that the medians of the blocks form a sorted sequence.

Step 4: *Trim blocks.* Let $X_1, X_2, \ldots, X_{n/\ell}$ denote the blocks in order of appearance after Step 3. For each pair of consecutive blocks X_i and X_{i+1}, $1 \leqslant i < n/\ell$, remove the same number of elements from the end of X_i as from the front of X_{i+1} such that

(i) X_iX_{i+1} is sorted and

(ii) if we remove fewer elements, X_iX_{i+1} is not sorted.

Step 5: *Sort removed elements.* Sort the elements that were removed in Step 4.

Step 6: *Merge.* Merge the sorted sequence left in X with the removed elements.

Denote by $R_X(\ell)$ the number of elements removed in Step 4. If $\ell > n/2$, $R_X(\ell)$ is defined to be 0. The following lemma shows how the performance of the Block Routine depends on ℓ and $R_X(\ell)$.

Lemma 2. *Block Routine* (X,ℓ) *can be implemented to run in time*

$$O\left(n\log\ell + \frac{n}{\ell}\cdot\log\left(\frac{n}{\ell}\right) + R_X(\ell)\log R_X(\ell)\right),$$

using $O(n)$ *extra space.*

Proof. Assume that when sorting in Steps 1, 2, and 5, any linear-space, worst-case optimal sorting algorithm is applied.

Step 1: *Sort blocks.* n/ℓ blocks of ℓ elements each can be sorted in $O(n\log\ell)$ time.

Step 2: *Sort medians.* There are n/ℓ medians, which take $O((n/\ell)\log(n/\ell))$ time to sort.

Step 3: *Permute blocks.* Permuting the blocks takes linear time by moving them one by one.

Step 4: *Trim blocks.* Let X_i and X_{i+1} be two blocks from which elements are to be removed. We seek the smallest j, $j\leqslant\ell/2$, such that removing the j greatest elements in X_i and the j smallest in X_{i+1}, makes the concatenation of what remains in X_i and X_{i+1} sorted. To do this we make two concurrent binary searches; one in the greater half of X_i and one in the smaller half of X_{i+1}, in the obvious way. This takes $O((n/\ell)\log\ell)$ time, which is linear. Further, compacting the removed and the remaining elements takes linear time as well.

Step 5: *Sort removed elements.* There are $R_X(\ell)$ elements to sort, which takes $O(R_X(\ell)\log R_X(\ell))$ time.

Step 6: *Merge.* This is an ordinary merge and runs in linear time.

Adding up gives the claimed bound. □

Before proceeding with a more thorough analysis of the Block Routine let us give some intuition. That is, why would we expect it to adapt to the number of inversions? Suppose there are few inversions in X. The key observation is that then $R_X(\ell)$ cannot be too large. The reason is that if d_i elements are removed to make X_iX_{i+1} a sorted sequence, then these elements caused at least $d_i^2/4$ inversions. In Lemma 4 below, we will show that $R_X(\ell)$ is decreasing on the block length ℓ. Hence, if we choose ℓ large enough, the time used by the Block Routine is dominated by the first two terms in the bound of Lemma 2. The goal in the following subsections is indeed to compute an appropriate ℓ.

3.1. Importance of the block length

In order to bound $R_X(\ell)$ from above we first show that Step 3 does not increase the number of inversions by more than a constant factor.

Lemma 3. $Inv(X_1X_2\cdots X_{n/\ell}) \leqslant 3 \cdot Inv(X)$.

Proof. Let X_i and X_j, $i < j$, be any two blocks in the permuted sequence. Consider all inversions caused by choosing one element from X_i and one element from X_j. We distinguish between the cases of whether X_i preceded X_j in the original sequence as well or if they changed order in Step 3. In the former case the number of inversions is not altered. In the latter X_i and X_j contributed at least $(|X_i|/2)\cdot(|X_j|/2) = \ell^2/4$ to $Inv(X)$. Step 3 undoes all these inversions, but it might introduce new ones. However, the number of new inversions is bounded by $|X_i|\cdot(|X_j|/2)+(|X_i|/2)\cdot(|X_j|/2) = 3\ell^2/4$, which is at most three times as many as the original amount. The claim now follows by considering all pairs of blocks. □

Observe that Lemma 3 holds regardless of whether we actually sort the blocks in Step 1 or just find the medians, since inversions *within* the blocks are not affected in Step 3.

We are now in a position to show how $R_X(\ell)$ decreases on the block length ℓ:

Lemma 4. $R_X(\ell) \leqslant \sqrt{12n \cdot Inv(X)/\ell}$.

Proof. Denote by d_i, $1 \leqslant i < n/\ell$, the number of elements that are removed to make X_iX_{i+1} a sorted sequence in Step 4. Then

$$R_X(\ell) = \sum_{i=1}^{n/\ell-1} d_i. \tag{1}$$

By Lemma 3 and since each d_i elements cause $d_i^2/4$ inversions we have

$$\sum_{i=1}^{n/\ell-1} \frac{d_i^2}{4} \leqslant 3 \cdot Inv(X). \tag{2}$$

$R_X(\ell)$ is maximized, subject to Eq. (2), if all d_i's are about the same. For simplicity assume that $R_X(\ell) = d\cdot(n/\ell-1)$, for some integer d. Then Eq. (2) can be expressed as

$$\left(\frac{n}{\ell}-1\right)\cdot\frac{d^2}{4} \leqslant 3 \cdot Inv(X),$$

which implies

$$d \leqslant \sqrt{\frac{12 \cdot Inv(X)}{n/\ell-1}}. \tag{3}$$

Combining Eqs. (1) and (3) gives an upper bound on the number of removed elements, namely

$$R_X(\ell) \leqslant \left(\frac{n}{\ell}-1\right)\cdot d \leqslant \sqrt{\frac{12n \cdot Inv(X)}{\ell}},$$

which completes the proof. □

Note that Lemma 4 only gives a worst-case upper bound on $R_X(\ell)$. There are sequences X for which $R_X(\ell) > R_X(\ell/2)$. For example, for

$$X = \langle 1, 2, \ldots, k, 3k+1, 3k+2, \ldots, 4k, k+1, k+2, \ldots, 3k \rangle,$$

of length $4k$, we have $R_X(2k) = 2k$ while $R_X(k) = 0$.

Let us *hypothesize* that we know the value of $Inv(X)$ a priori. Then, using Lemma 4, we can pick ℓ such that

$$R_X(\ell) \log R_X(\ell) = O\left(n \log \ell + \frac{n}{\ell} \cdot \log\left(\frac{n}{\ell}\right)\right),$$

i.e., the time required by the Block Routine (Lemma 2) becomes $O(n \log \ell + (n/\ell) \log(n/\ell))$. For instance, suppose $Inv(X) = n(\log n)^2$. (According to Lemma 1, to be optimal, X should be sorted in time $O(n \log \log n)$.) Then, putting $\ell = (\log n)^4$ guarantees that $R_X(\ell) = O(n/\log n)$, and the total running time becomes $O(n \log \log n)$.

Note that, had we picked $\ell = (\log n)^3$, Lemma 4 would only have guaranteed $R_X(\ell) = O(n/\sqrt{\log n})$, which would have given a running time of $O(n\sqrt{\log n})$, which is too much. On the other hand, had we picked a larger ℓ, e.g. $\ell = n^{1/4}$, we would have had $R_X(\ell) = o(n/\log n)$; however, in this case the $n \log \ell$ term in the total running time contributes $\Theta(n \log n)$, which is too much.

Given $Inv(X)$, we can thus choose an appropriate block length, but in practice we can certainly not assume that this quantity is part of the input. Then, how do we choose ℓ? This is the topic of the next subsection.

3.2. Computing the block length

The main tool when searching for an appropriate ℓ is that of computing $R_X(\ell)$ fast, given a value of ℓ; this is dealt with in Lemma 5. The search is then accomplished by, starting from a small value, performing a doubly exponential search for an ℓ such that $R_X(\ell) \leqslant n/\log n$ (and consequently $R_X(\ell) \log R_X(\ell) = O(n)$). Lemma 7 analyzes the searching algorithm, and Lemma 8 shows that using the found value as block length in the Block Routine leads to *Inv*-optimality.

For reasons that we will return to in Section 3.3, at this point we restrict ourselves to block lengths no smaller than $(\log n)^3$.

Lemma 5. *Given ℓ, $\ell \geqslant (\log n)^3$, $R_X(\ell)$ can be computed in linear time.*

Proof. Our method relies on the fact that the median of n elements can be computed in linear time [21]. For simplicity, we assume that as part of the process of finding the median, we get a partitioning of the input array into two parts; one consisting of all elements smaller than the median and one with the remaining elements. Essentially, we perform Step 1, Step 2, and Step 4 of the Block Routine.

Rather than sorting the blocks in Step 1 we just compute the median in each block in $O(\ell) \cdot n/\ell = O(n)$ time in total. Since $\ell \geqslant (\log n)^3$, sorting the medians in Step 2 takes sublinear time.

As in the proof of Lemma 4, let d_i denote the number of elements that need to be removed in Step 4 to make X_iX_{i+1} a sorted sequence. No removals are carried out but we just compute the d_i's. Note that we are not supplied with sorted blocks as was the case in the Block Routine. Nevertheless, a method similar to the one given in the proof of Lemma 2 is applicable. We perform two concurrent binary searches; one in the greater half of X_i and one in the smaller half of X_{i+1}, by repeated median finding. That is, we compute the medians in the greater half of X_i and the smaller half of X_{i+1}. Depending on the outcome of a comparison between these two medians we then proceed recursively with either the greatest fourth of X_i and the smallest fourth of X_{i+1}, or the second greatest fourth of X_i and the second smallest fourth of X_{i+1}. The time needed to compute d_i is given by

$$T(\ell) \leqslant \begin{cases} O(1) & \text{if } \ell \leqslant 1, \\ T(\ell/2) + O(\ell) & \text{otherwise,} \end{cases}$$

which has the solution $T(\ell) = O(\ell)$. Summing over all d_i's and adding the cost for the earlier steps gives the claimed bound. □

Consider now the following algorithm for determining ℓ:

```
function Find-length (X : sequence): integer
  ℓ := (log n)^3
  repeat
    ℓ := ℓ^2
  until ℓ > n/2 or R_X(ℓ) ≤ n/log n
  if ℓ > n/2 then
      return n
    else
      return ℓ
end
```

When computing $R_X(\ell)$ it is important that the algorithm applied does not move elements between the blocks but only permutes the elements within the blocks. Otherwise the computation of $R_X(\ell)$ could affect the value of $R_X(\ell^2)$, which is invariant under permutations within the blocks of length ℓ.

The following technical corollary of Lemma 4 deals with the 'base case' of Find-length:

Corollary 6. *If* $R_X((\log n)^3) > n/\log n$ *then* $Inv(X) = \Omega(n \log n)$.

Proof. Setting $\ell = (\log n)^3$ in the upper bound given in Lemma 4 yields

$$\frac{n}{\log n} < R_X((\log n)^3) \leqslant \sqrt{\frac{12n \cdot Inv(X)}{(\log n)^3}},$$

from which it follows that $Inv(X) = \Omega(n \log n)$. □

Recall that we are eventually aiming for a sorting algorithm that runs in time $O(n \log(Inv(X)/n))$. One way of interpreting Corollary 6 arises when it is combined with Lemma 1; namely, if $R_X((\log n)^3) > n/\log n$ then a sorting algorithm which runs in $(n \log\log n)$ time is *Inv*-optimal.

The next lemma analyzes the behavior of Find-length:

Lemma 7. *Suppose* $R_X((\log n)^3) > n/\log n$.

- *If* $\ell = Find\text{-}length(X) < n$ *then* $R_X(\sqrt{\ell}) > n/\log n$ *and* $R_X(\ell) \leqslant n/\log n$.
- *If* $Find\text{-}length(X) = n$ *then* $Inv(X) = \Omega(n^{3/2}/(\log n)^2)$.
- $Find\text{-}length(X)$ *runs in time* $O(n \log\log(Inv(X)/n))$.

Proof. Consider the repeat-loop in the algorithm. If we exit due to the second condition then $R_X(\ell) \leqslant n/\log n$. We must also have $R_X(\sqrt{\ell}) > n/\log n$, or we would have exited in the previous iteration.

If $Find\text{-}length(X) = n$ then we exited due to the first condition, that is, $\ell > n/2$. Then the second condition was not fulfilled in the previous iteration, so $R_X(\sqrt{\ell}) > n/\log n$. Now, as $\ell > n/2$ it follows that $\sqrt{\ell} > \sqrt{n/2}$, and consequently $R_X(\sqrt{n/2}) > n/\log n$. We combine this with Lemma 4 to obtain

$$\frac{n}{\log n} < R_X(\sqrt{n/2}) \leqslant \sqrt{\frac{12n \cdot Inv(X)}{\sqrt{n/2}}},$$

from which it follows that $Inv(X) = \Omega(n^{3/2}/(\log n)^2)$.

In proving the bound on the time consumed by $Find\text{-}length(X)$, we note that each iteration takes linear time by Lemma 5. It thus suffices to show that the number of iterations of the repeat-loop is $O(\log\log(Inv(X)/n))$. Further, observe that the values on ℓ for which $R_X(\ell)$ is computed are on the form $(\log n)^{3 \cdot 2^i}$, where i is an integer.

We start by verifying the claim when the algorithm does not find an appropriate ℓ, that is, when n is returned. Since $(\log n)^{3 \cdot 2^i} > n/2$ for any $i \geqslant \log\log n$, the number of iterations of the repeat-loop is bounded by $\log\log n$ which is $O(\log\log(Inv(X)/n))$ if $Inv(X) = \Omega(n^{3/2}/(\log n)^2)$.

Next, consider the case when $Find\text{-}length(X) < n$. Using Lemma 4 it is an easy matter to show that if

$$i \geqslant \log\left(\log\left(\frac{12 \cdot Inv(X)}{n}\right) + 2\log\log n\right)$$

then $R_X((\log n)^{3 \cdot 2^i}) \leqslant n/\log n$. Next, by Corollary 6 and the assumption $R_X((\log n)^3) > n/\log n$, we have $Inv(X) = \Omega(n \log n)$. Therefore

$$\log\left(\log\left(\frac{12 \cdot Inv(X)}{n}\right) + 2\log\log n\right) = O\left(\log\log\left(\frac{Inv(X)}{n}\right)\right),$$

which completes the proof. □

The next lemma demonstrates that using the block length returned by Find-length in the Block Routine indeed results in an *Inv*-optimal sorting algorithm:

Lemma 8. *Let* $R_X((\log n)^3) > n/\log n$. *Block Routine*$(X,$ *Find-length*$(X))$ *runs in time* $O(n \log(Inv(X)/n))$.

Proof. Recall that, by Lemma 2 the time spent by the Block Routine (X,ℓ) is bounded by

$$O\left(n \log \ell + \frac{n}{\ell} \log\left(\frac{n}{\ell}\right) + R_X(\ell) \log R_X(\ell)\right),$$

Consider first the case when $\ell = \textit{Find-length}(X) < n$. Then this is bounded by $O(n \log \ell)$, because $\ell > (\log n)^3$ and $R_X(\ell) \leqslant n/\log n$, by Lemma 7. It remains to prove that $n \log \ell = O(n \log(Inv(X)/n))$. To do this we combine the fact that $R_X(\sqrt{\ell}) > n/\log n$ and Lemma 4 to establish an upper bound on ℓ:

$$\frac{n}{\log n} < R_X(\sqrt{\ell}) \leqslant \sqrt{\frac{12n \cdot Inv(X)}{\sqrt{\ell}}},$$

implying that

$$\ell = O\left(\left(\frac{Inv(X)}{n} \cdot (\log n)^2\right)^2\right).$$

Hence

$$n \log \ell = O\left(n \log\left(\frac{Inv(X)}{n}\right) + n \log\log n\right).$$

But as $R_X((\log n)^3) > n/\log n$, Corollary 6 gives that $Inv(X) = \Omega(n \log n)$, and thus

$$n \log\log n = O\left(n \log\left(\frac{Inv(X)}{n}\right)\right),$$

which completes the proof. □

If $\ell = n$ the time consumed is $O(n \log n)$, which is $O(n \log(Inv(X)/n))$, since in this case $Inv(X) = \Omega(n^{3/2}/(\log n)^2)$, by Lemma 7.

3.3. Putting things together

Consider the following algorithm for sorting a sequence X of length n.

```
procedure Blocksort (X: sequence)
  if R_X((log n)^3) ⩽ n/log n then
        Block Routine(X, (log n)^3)
    else
      Block Routine(X, Find-length(X))
end
```

Lemma 2 ensures that if we end up in the then-branch, the sort takes $O(n \log\log n)$ time. Otherwise, it takes $O(n \log(Inv(X)/n))$ time, according to Lemma 8. We have thus proved

Lemma 9. *Blocksort sorts a sequence X of length n in time*

$$O\left(n \log\left(\frac{Inv(X)}{n}\right) + n \log\log n\right),$$

which is optimal with respect to Inv if $Inv(X) = \Omega(n(\log n)^{\varepsilon})$, for any constant $\varepsilon > 0$. The amount of space used is $O(n)$.

Proof. The optimality follows from Lemma 1. □

At this point it is natural to ask how to obtain *Inv*-optimality when there are few inversions in the input. Is it not possible to choose the block length so that we end up with an optimal algorithm in this case too? Recall the time consumed by the Block Routine as a function of ℓ, as stated in Lemma 2:

$$O\left(n \log \ell + \frac{n}{\ell} \cdot \log\left(\frac{n}{\ell}\right) + R_X(\ell) \log R_X(\ell)\right).$$

It is an easy matter to verify that the first two terms add up to $\Omega(n \log\log n)$ for any ℓ, $1 \leqslant \ell \leqslant n$. Hence, the presented implementation of the Block Routine requires $\Omega(n \log\log n)$ time independently of how the block length is chosen.

One way of overcoming this problem is to apply the Block Routine recursively to sort the blocks. This would lead to a total running time of $O(n \log(Inv(X)/n) + n \log^* n)$, where $\log^* n$ denotes the number of applications of the log-function required to reduce n to a value not greater than 1. In the next section, we show how to avoid the $n \log^* n$ term by stopping the recursion early and applying an existing *Inv*-optimal algorithm to sort the subblocks.

4. The in-place algorithm

We present a sorting algorithm which is optimal with respect to the number of inversions and uses only constant extra space. Essentially, Blocksort is implemented to run in-place. However, in the previous section we learned that Blocksort takes $\Omega(n \log\log n)$ time, which is not *Inv*-optimal if $Inv(X) = o(n(\log n)^{\varepsilon})$, $\varepsilon > 0$, and hence another method is needed to take care of such inputs. The main scheme includes first sorting all blocks of length $(\log n)^3$ in $O(n \log(Inv(X)/n))$ time and then applying the Block Routine to the whole sequence. More formally, we proceed as follows:

procedure *In-place Blocksort* (X : *sequence*)
  Divide X into $n/(\log n)^3$ equal-length blocks, $X_1, \ldots, X_{n/(\log n)^3}$
  for $i := 1$ **to** $n/(\log n)^3$ **do**

Block Routine(X_i, $\log n/\log\log n$)
if $R_X((\log n)^3) \leqslant n/\log n$ **then**
Perform Steps 2–6 of *Block Routine*(X, $(\log n)^3$)
else
Block Routine(X, *Find-length*(X))
end

The remainder of this section proves that the above algorithm can be implemented to run in-place and *Inv*-optimal. The next subsection is devoted to the technique of encoding pointers in sorted arrays, which is a crucial tool in our algorithm. In Section 4.2 we show how to implement the Block Routine in constant extra space without increasing its asymptotic time complexity. In Section 4.3 we provide the remaining details needed for sorting the blocks, which when combined with the result in the preceding subsection implies our main theorem.

4.1. Encoding pointers

Using a sorted array to encode pointers is a fairly well-known trick when designing implicit data structures; see e.g. [22]. For now, assume that all elements are distinct. Following Munro [22], a pointer with a value between 0 and $n-1$ is encoded in the relative order of the elements in a block of length $2\lceil\log n\rceil$. The elements are almost stored in sorted order; however, every pair of consecutive elements starting at an odd position is in increasing order to represent a 0 and reversed to denote a 1. Reading all bits encoded in a block thus gives the binary representation of an integer between 0 and $n-1$. It is clear that this encoding of pointers supports reading and updating in $O(\log n)$ time. Note that the requirement that all elements should be distinct is not actually needed. The technique works as long as every consecutive pair of elements used contain two distinct elements. Hence, using a sorted array is in fact the least smart thing to do, because then the probability that a pair contains two identical elements is maximized. In a moment we will see that the elements' distinctness is not necessary at all.

In our algorithm $2n/(\log n)^3$ pointers are needed, requiring $2n/(\log n)^2$ pairs of elements. We start by sorting the first $4n/(\log n)^2$ elements of the input by Heapsort, which takes sublinear time and $O(\log n)$ bits of extra space. Next, make a perfect shuffle of the first and the second half of the sorted portion. (This is easily accomplished in $o(n)$ time using $O(\log n)$ bits of extra space by swapping every element appearing at an even position in the first half with the element at the corresponding position in the second half.) If no element among the sorted ones appears more than $2n/(\log n)^2$ times, every pair of consecutive elements now contains distinct elements. If there is an element x which is too frequent, however, changes have to be made. One simple way of doing this is to search forward in the array until we have found in total $2n/(\log n)^2$ elements which are different from x. If there are not that many elements different from x, we need not bother about *Inv*-optimality, but it suffices to collect the x's in one part

of the array, sort the remaining part by Heapsort, and finally merge using a linear-time, in-place merging algorithm [13]. This will take $\Theta(n)$ time and constant extra space in total. (A similar technique was used by Lai and Wood [16] in the design of an in-place selection algorithm.)

During the entire algorithm the first portion of the array will be used as a pointer encoding area. When the remaining portion is eventually sorted we restore the sorted order in this portion and merge the two portions by a linear-time, in-place merging algorithm. In the sequel it is helpful to think of the pointer encoding area as an array with $2n/(\log n)^3$ entries, each capable of storing a pointer, and with $O(\log n)$ cost for accesses and updates.

4.2. *In-place implementation of the Block Routine*

We describe how all steps but the first of the Block Routine can be implemented to run in-place. At first, it might seem that this is easily done by applying Heapsort whenever sorting and using a linear-time, in-place merging algorithm. Apart from Step 3 this is basically true. To carry out this step, however, requires knowledge of the permutation, which is where the encoding of pointers comes into play.

Step 2: *Sort medians.* Assume that the blocks of length $\ell \geqslant (\log n)^3$ are sorted and consider the problem of sorting the medians of the blocks in linear time and constant extra space. The n/ℓ medians are sorted by Heapsort, where the heap consists of the entries in the input array which hold the medians, that is, every ℓth position starting from the position in which the first median is located. To store the permutation we keep track of the original position for each of the medians, as follows. Logically, the first half of the pointer encoding area is a heap as well. For each median in the heap, the corresponding entry in the pointer encoding area contains the address of the median's original position. Hence, each swap performed by Heapsort consists also of a swap of two pointers in the pointer encoding area. As Heapsort is worst-case optimal and since swapping two pointers takes $O(\log n)$ time, the time used in Step 2 is

$$O\left(\log n \cdot \frac{n}{\ell} \log\left(\frac{n}{\ell}\right)\right) = O\left(\frac{n}{\log n}\right),$$

because $\ell \geqslant (\log n)^3$. Before permuting the blocks in Step 3 we restore the order by moving all medians back to their original positions, according to the addresses in the pointer encoding area, which takes sublinear time.

Step 3 : *Permute blocks.* Consider now the task of permuting the blocks in Step 3. In the following, let us identify the address to a block with the address to the median of the block. It is important to observe that the pointer encoding area holds a permutation π of $\{1,\ldots,n/\ell\}$, where $\pi(i)$, that is, the value of the ith pointer, is the address to the block X_i, that is, the ith block in the sought sequence. To permute the blocks within the required resource bounds we need the inverse permutation π^{-1} rather than π itself. Note that π^{-1} is the permutation such that the block $X_{\pi^{-1}(j)}$ is the jth block in the present order. Equivalently, a block which is currently the jth will end up

as the $\pi^{-1}(j)$th. We use the second half of the pointer encoding area for storing π^{-1}. Given π, the following statement computes π^{-1}:

for $i := 1$ **to** n/ℓ **do** $\pi^{-1}(\pi(i)) := i$

It is now easy to permute the blocks (cf. sorting a permutation of $\{1,\ldots,n\}$):

for $i := 1$ **to** n/ℓ **do**
  while $\pi^{-1}(i) \neq i$ **do**
    Swap the blocks located at positions i and $\pi^{-1}(i)$
    Swap $\pi^{-1}(i)$ and $\pi^{-1}(\pi^{-1}(i))$
  endwhile

Since each swap of two blocks moves a block to its correct position the total number of elements moved is linear. Further, a pointer is read or updated only when a block is moved, that is, $O(n/\ell)$ times. Again, as $\ell \geqslant (\log n)^3$ and since accessing a pointer costs $O(\log n)$ time the time taken due to operations on pointers is sublinear. We conclude that Step 3 can be carried out in linear time and constant extra space.

Step 4: *Trim blocks.* From the proof of Lemma 2 we recall that for two consecutive blocks, X_i and X_{i+1}, computing the number of elements that need to be removed, d_i, was accomplished by binary searches, which run in-place. Performing the actual removals is done by pushing the elements removed so far in front of us. That is, after the first removal, the first $2\ell - d_1$ positions contain the (sorted) concatenation of what remains in X_1 and X_2, and the following d_1 positions contain the d_1 removed elements. In general, after the k:th removal, the first $(k+1)\ell - \sum_{i=1}^{k} d_i$ positions hold the concatenation of what remains in the blocks $X_1, X_2, \ldots, X_k$, while the remaining part of the array consists of all elements removed so far followed by blocks that have not yet been considered. When finished, the array consists of a sorted sequence of length $n - R_X(\ell)$ followed by the $R_X(\ell)$ removed elements. Since each swap puts an element in its correct position the total time spent is linear. Further, no extra space was needed.

Step 5: *Sort removed elements.* Heapsort can be applied to sort the last unsorted portion of the array in $O(R_X(\ell) \log R_X(\ell))$ time and constant extra space.

Step 6: *Merge.* A linear-time, in-place merging algorithm can be applied.

Adding up the above time bounds for the various steps gives

Lemma 10. *Let $\ell \geqslant (\log n)^3$ and assume that the blocks are sorted. Then the Block Routine can be implemented to run in $O(n + R_X(\ell) \log R_X(\ell))$ time and constant extra space.*

Recall from the proof of Lemma 5 that, given ℓ, $\ell \geqslant (\log n)^3$, $R_X(\ell)$ is computed by performing Step 2 and parts of Step 4 of the Block Routine combined with median findings. Replacing the selection algorithm by the linear-time, in-place selection algorithm due to Lai and Wood [16], and performing Step 2 and Step 4

in-place, as discussed above, ensures that Lemma 5 holds for an in-place algorithm as well. As an immediate consequence, the function Find-length given in Section 3.2 can be implemented using constant extra space without increasing the time consumed by more than a constant factor. Hence, Lemma 7 holds also for an in-place algorithm.

Before summarizing the above discussion, recall the description of In-place Blocksort. We have yet to show how to sort the blocks. Denote by $T(X_i)$ the time needed to sort the i:th block of length $(\log n)^3$. Then the first statement of In-place Blocksort takes time $\sum_{i=1}^{n/(\log n)^3} T(X_i)$. If we in the else-branch of In-place Blocksort apply Heapsort when sorting the blocks, the if-statement in our algorithm becomes almost identical to the algorithm given in Section 3.3. The difference is that if $R_X((\log n)^3) \leqslant n/\log n$, the blocks are already sorted. Since Lemmas 5 and 7 hold for in-place algorithms, by the above discussion, the same arguments as in Section 3.3 imply a lemma similar to Lemma 9, namely

Lemma 11. *In-place Blocksort sorts a sequence X of length n in time*

$$O\left(n \log\left(\frac{Inv(X)}{n}\right)\right) + \sum_{i=1}^{n/(\log n)^3} T(X_i),$$

using constant extra space plus the space needed when sorting the blocks.

It thus remains to prove that $\sum_{i=1}^{n/(\log n)^3} T(X_i) = O(n \log(Inv(X)/n))$, which is what is done in the next subsection.

4.3. Sorting the blocks

We apply Lemma 10 to sort each block X_i of length $(\log n)^3$ using the Block Routine with subblock length $\ell = \log n/\log\log n$. First note that for sufficiently large n, $\log n/\log\log n \geqslant \log((\log n)^3)$, and thus Lemma 10 can indeed be applied if we show how to sort the subblocks. We show how to sort each subblock X_i in $O((\log n)^3 \cdot \log(Inv(X_i)/(\log n)^3))$ time and constant extra space. The desired result then follows by the following lemma.

Lemma 12. *Let X be divided into blocks of equal length ℓ. If each block X_i is sorted in time $O(\ell \log(Inv(X_i)/\ell))$, the total amount of time spent is $O(n \log(Inv(X)/n))$.*

Proof. The time consumed is proportional to

$$\sum_{i=1}^{n/\ell} \ell \log\left(\frac{Inv(X_i)}{\ell}\right) = \ell \log\left(\prod_{i=1}^{n/\ell} \frac{Inv(X_i)}{\ell}\right)$$

$$= \ell \log\left(\prod_{i=1}^{n/\ell} \left(\frac{Inv(X_i)}{\ell}\right)^{\ell/n}\right)^{n/\ell}$$

$$= n \log \left(\prod_{i=1}^{n/\ell} \left(\frac{Inv(X_i)}{n} \right)^{\ell/n} \right)$$

$$\leqslant n \log \left(\sum_{i=1}^{n/\ell} \frac{Inv(X_i)}{n} \right)$$

$$\leqslant n \log \left(\frac{3\, Inv(X)}{n} \right),$$

where the inequality follows from the inequality relating the geometric and arithmetic averages. □

According to Lemma 10 it remains to show how to implement Step 1, and that the sort of the removed elements in Step 5 can be accomplished fast enough.

Step 1: *Sort subblocks.* Consider the task of sorting each subblock of length $\log n/\log\log n$. (No encoding of pointers is used in this step). This is accomplished by applying the *Inv*-optimal sorting algorithm Splitsort [18]. Presented with an array of length ℓ, Splitsort uses only ℓ pointers in addition to the input array. Therefore, since each pointer requires only $O(\log \ell)$ bits and since, in our case, $\ell = \log n/\log\log n$ the number of extra bits needed is

$$O(\ell \log \ell) = O\left(\frac{\log n}{\log\log n} \cdot \log \left(\frac{\log n}{\log\log n} \right) \right) = O(\log n).$$

Setting $k = \log n/\log\log n$ and replacing n by $(\log n)^3$ in Lemma 12 gives that the total time consumed by the $(\log n)^2 \cdot \log\log n$ calls to Splitsort is $O((\log n)^3 \cdot \log(Inv(X_i)/(\log n)^3))$.

Step 5: *Sort removed elements.* Let $R_{X_i}(\log n/\log\log n)$ be the number of elements removed in Step 4 of the Block Routine when applied to sort the block X_i with $\ell = \log n/\log\log n$. Then Step 5 runs in time

$$T(R_{X_i}) = O\left(R_{X_i}\left(\frac{\log n}{\log\log n} \right) \cdot \log \left(R_{X_i}\left(\frac{\log n}{\log\log n} \right) \right) \right).$$

Replacing n by $(\log n)^3$ and using $\ell = \log n/\log\log n$ in Lemma 4 gives

$$R_{X_i}\left(\frac{\log n}{\log\log n} \right) \leqslant \sqrt{\frac{12(\log n)^3 \cdot Inv(X_i)}{\log n/\log\log n}}$$

$$= O\left(\sqrt{Inv(X_i) \cdot (\log n)^2 \cdot \log\log n} \right).$$

Observing that $Inv(X_i) = O((\log n)^6)$ yields

$$T(R_{X_i}) = O\left(\sqrt{Inv(X_i) \cdot (\log n)^2 \cdot (\log\log n)^3} \right).$$

Furthermore, since $R_{X_i}(\log n/\log\log n)$ cannot possibly exceed the total number of elements in X_i, that is, $(\log n)^3$, we have

$$T(R_{X_i}) = O\left(\min\left\{ (\log n)^3 \cdot \log\log n, \sqrt{Inv(X_i) \cdot (\log n)^2 \cdot (\log\log n)^3} \right\} \right). \quad (4)$$

Recall that we aim to prove that $T(R_{X_i}) = O((\log n)^3 \cdot \log(Inv(X_i)/(\log n)^3))$. We distinguish two cases. First, if $Inv(X_i) \geqslant (\log n)^{7/2}$ then

$$(\log n)^3 \cdot \log\left(\frac{Inv(X_i)}{(\log n)^3}\right) = \Omega((\log n)^3 \cdot \log\log n),$$

and hence the first term of Eq. (4) is small enough. Otherwise, if $Inv(X_i) < (\log n)^{7/2}$ is inserted into the second term of Eq. (4) we get

$$\begin{aligned} T(R_{X_i}) &= O\left(\sqrt{Inv(X_i) \cdot (\log n)^2 \cdot (\log\log n)^3}\right) \\ &= O\left((\log n)^{11/4} \cdot (\log\log n)^{3/2}\right), \end{aligned}$$

which is $O((\log n)^3)$ and therefore $O((\log n)^3 \cdot \log(Inv(X_i)/(\log n)^3))$. Hence, in either case $T(R_{X_i}) = O((\log n)^3 \cdot \log(Inv(X_i)/(\log n)^3))$.

We have thus proved that each block X_i of length $(\log n)^3$ can be sorted in time $O((\log n)^3 \cdot \log(Inv(X_i)/(\log n)^3))$ using only a constant number of extra words. Setting $\ell = (\log n)^3$ and applying Lemma 12 now yields

Lemma 13. *To sort all blocks of length* $(\log n)^3$ *using constant extra space takes time*

$$\sum_{i=1}^{n/(\log n)^3} T(X_i) = O\left(n \log\left(\frac{Inv(X)}{n}\right)\right).$$

Combining Lemmas 1, 11, and 13 implies the main result of this section:

Theorem 14. *In-place Blocksort is optimal with respect to the number of inversions and uses* $O(\log n)$ *bits of extra space.*

Next, consider the measure *Rem*, defined as

$$Rem(X) = n - \max\{m \mid X \text{ has an ascending subsequence of length } m\}.$$

Apart from *Inv* and the number of ascending runs, *Rem* is the most commonly used measure of presortedness; see e.g. [15, 17, 18, 20].

Theorem 15. *In-place Blocksort sorts any sequence* X *of length* n *with* $Rem(X) = k$ *in* $O(n + k \log k)$ *time, which is optimal with respect to Rem.*

Proof. Let X be a sequence of length n with $Rem(X) = k$. In [17] we proved that $R_X(\ell) = O(k)$, independently of the value of ℓ. Moreover, Splitsort uses at most $O(n + k \log k)$ time [18]. It follows that In-place Blocksort sorts X in $O(n + k \log k)$ time, by Lemma 10. The optimality was proved by Mannila [20]. □

5. Practical algorithms

Recall the algorithm Blocksort given in Section 3.3. Even though the time taken for computing the appropriate block length is asymptotically dominated, its implicit constant factor is considerable compared to that of the leading term; the reason being the repeated median finding. To date there is no known worst-case optimal (that is, linear time) selection algorithm which is so fast that one would consider implementing it as a critical part of a sorting algorithm. One could of course settle for a simple algorithm that is fast on the average, such as the one by Floyd and Rivest [9]. For our purposes, the best would be if we knew which block length to choose beforehand!

5.1. A wishful thought

Suppose that when presented with a sequence X to sort we are also informed of the number of inversions in X. Could we use this extra information to decide upon an optimal block length? Consider the following simple variant of Blocksort:

procedure *Informed Blocksort* (X:*sequence*; *Inv*(X): *integer*)
  Divide X into $n/(\log n)^3$ equal-length blocks, $X_1,\ldots,X_{n/(\log n)^3}$
  for $i := 1$ **to** $n/(\log n)^3$ **do**
    Heapsort(X_i)
  if $R_X((\log n)^3) \leqslant 2n/\log n$ **then**
    Perform Steps 2–6 of *Block Routine*($X,(\log n)^3$)
    else
    $$\textit{Block Routine}\left(X, \min\left\{n, \max\left\{(\log n)^3, \frac{\textit{Inv}(X)\cdot(\log n)^2}{n}\right\}\right\}\right)$$
end

We proceed by analyzing the time consumed by the above algorithm. Since sorting the blocks of length $(\log n)^3$ takes $O(n \log\log n)$ time Informed Blocksort runs in $O(n \log\log n)$ time if $R_X((\log n)^3) \leqslant 2n/\log n$, by Lemma 2.

Otherwise, if $R_X((\log n)^3) > 2n/\log n$, it follows by Corollary 6 that $Inv(X) > n \log n$. We claim that in this case Informed Blocksort runs in time $O(n \log(Inv(X)/n))$. We distinguish three cases depending on the value assigned to ℓ in the else-branch of the algorithm:

$\ell = n$: Then the algorithm runs in $\Theta(n \log n)$ time since it degenerates to a worst-case optimal sort of the whole sequence. But if $\ell = n$ then $Inv(X)\cdot(\log n)^2/n \geqslant n$, that is, $Inv(X) \geqslant n^2/(\log n)^2$, and hence $n \log(Inv(X)/n) = \Theta(n \log n)$.

$\ell = (\log n)^3$: Then we must have $Inv(X) > n \log n$, and Lemma 4 implies

$$R_X(\ell) \leqslant \sqrt{\frac{12n^2 \log n}{(\log n)^3}} = O\left(\frac{n}{\log n}\right).$$

Hence, by Lemma 2 the call to the Block Routine uses $O(n \log\log n)$ time, which is $O(n \log Inv(X)/n))$ since $Inv(X) > n \log n$.

$\ell = Inv(X) \cdot (\log n)^2/n$: Then by Lemma 4

$$R_X(\ell) \leqslant \sqrt{\frac{12n \cdot Inv(X)}{Inv(X) \cdot (\log n)^2/n}} \leqslant \frac{\sqrt{12} \cdot n}{\log n}.$$

Substituting for ℓ and $R_X(\ell)$ in Lemma 2 gives that the call to the Block Routine totals $O(n \log(Inv(X)/n))$ time in this case as well.

We have thus proved that Informed Blocksort runs in time

$$O\left(n \log\left(\frac{Inv(X)}{n}\right) + n \log\log n\right),$$

and hence achieves the same bounds as Blocksort (cf. Lemma 9). We conclude that if we have a priori knowledge of the number of inversions and if this number is not too small, we can take advantage of it and design a simple adaptive sorting algorithm.

5.2. Randomization helps

We show that taking a random sample of linear size results in an appropriate estimation of the number of inversions in a sequence. Using this estimation gives us a practical algorithm which is as fast as Informed Blocksort with high probability.

If we randomly choose a pair of elements from a sequence X of length n, the probability that this particular pair is an inversion is

$$p = Inv(X) \Big/ \binom{n}{2}.$$

Assume that we repeat this k times, that is, we perform k Bernoulli trials. Letting S_k denote the number of inversions that we found, the probability that $S_k = r$, $r \leqslant k$, is given by the binomial distribution:

$$\Pr[S_k = r] = \binom{k}{r} p^r (1-p)^{k-r}.$$

Naturally, our hypothesis is that S_k/k is a good approximation of p. Slight manipulations of the Chernoff bounds on the random variable S_k, derived by Hagerup and Rüb [11, Eq. (12)], give

$$\Pr[S_k \geqslant (1+\varepsilon)kp] \leqslant \left[\frac{e^{\varepsilon}}{(1+\varepsilon)^{1+\varepsilon}}\right]^{kp}, \quad \varepsilon \geqslant 0, \tag{5}$$

$$\Pr\left[S_k \leqslant \frac{kp}{1+\delta}\right] \leqslant \left[\frac{1+\delta}{e^{\delta}}\right]^{\frac{kp}{1+\delta}}, \quad \delta \geqslant 0. \tag{6}$$

Setting $\varepsilon = 4$ and $\delta = 4$ in Eqs. (5) and (6), respectively, yields

$$\Pr\left[\frac{kp}{5} < S_k < 5kp\right] \geqslant 1 - \left(\left[\frac{e^4}{5^5}\right]^{kp} + \left[\frac{5^{1/5}}{e^{4/5}}\right]^{kp}\right)$$

$$\geqslant 1 - \left(\frac{1}{2}\right)^{kp/2} \tag{7}$$

Inserting $p = Inv(X)/\binom{n}{2}$ in Eq. (7) gives

$$\Pr\left[\frac{Inv(X)}{5} < \frac{S_k}{k} \cdot \binom{n}{2} < 5\,Inv(X)\right] \geqslant 1 - \left(\frac{1}{2}\right)^{k \cdot Inv(X)/n^2} \tag{8}$$

Finally, by letting $k = n$ in Eq. (8) we conclude

Theorem 16. *Let X be a sequence of length n. There is a linear time randomized algorithm which estimates the number of inversions in X to within a constant factor with probability at least $1 - 2^{-Inv(X)/n}$.*

Let $Inv^*(X)$ denote the estimated value of $Inv(X)$ obtained when taking a sample of size n, that is,

$$Inv^*(X) = \frac{S_n}{n} \cdot \binom{n}{2}.$$

Consider using $Inv^*(X)$ instead of the exact value of $Inv(X)$ in Informed Blocksort.

As noted in the previous subsection, the algorithm takes $O(n \log\log n)$ time if $R_X((\log n)^3) \leqslant 2n/\log n$; otherwise, if $R_X((\log n)^3) > 2n/\log n$, we have $Inv(X) > n \log n$. But if the number of inversions is this high our approximation is good with high probability: Setting $k = n$ in Eq. (8) implies

$$\Pr\left[\frac{Inv(X)}{5} < Inv^*(X) < 5\,Inv(X)\right] \geqslant 1 - \left(\frac{1}{2}\right)^{Inv(X)/n} \geqslant 1 - \frac{1}{n},$$

because $Inv(X) > n \log n$. Consequently, the probability that the block length computed as a function of the number of inversions is accurate up to a constant factor is at least $1 - 1/n$. It is now an easy matter to verify that the same asymptotic time bounds that were proved for Informed Blocksort in the previous subsection also hold for the randomized variant with high probability. That is, it runs in time $O(n \log(Inv(X)/n) + n \log\log n)$ with probability at least $1 - 1/n$.

From Eq. (8) we note that the probability that our estimation is more than a factor 5 from the correct value decreases exponentially with the size of the sample. For instance, if we take a sample of size cn rather than n, the time bound of the resulting sorting algorithm holds with probability at least $1 - 1/n^c$. Since the randomized Blocksort always takes $\Omega(n \log\log n)$ time, we could even increase the size of the sample to $n \log\log n$ resulting in that the time bounds hold with overwhelming probability.

6. Exploiting many inversions

As stated in the last paragraph in the previous section adaptive sorting algorithms generally have to pay something for being adaptive and the price shows up in the

implicit constant factors. Mehlhorn [21, p. 224] claims that the *Inv*-optimal sorting algorithm A-Sort is 'better' than Quicksort if $Inv(X) \leqslant n^{1.58}/50$. However, since the expected number of inversions in a sequence of length n is $n^2/4$, A-Sort requires significantly fewer inversions than is expected for surpassing Quicksort. All known *Inv*-optimal sorting algorithms have an implicit constant factor greater than one in the leading term of the number of comparisons performed [18, 19, 20]. Straightforward calculations give that as a consequence, they all require $Inv(X) = O(n^{2-\varepsilon})$, for some constant $\varepsilon > 0$, in order to perform less comparisons than does for example Mergesort.

The above discussion suggests the following question:

> How few inversions does there have to be in a sequence for it to be possible to sort by less comparisons than stated in the information-theoretic average-case lower bound for comparison-based sorting algorithms?

That is, how small degree of presortedness can we take advantage of? The remainder of this section is devoted to proving a theorem which states that the number of inversions can be asymptotically maximized, namely

Theorem 17. *There is a constant c and a sorting algorithm that sorts any sequence X of length n in $O(n \log n)$ time and less than $\log(n!)$ comparisons if $Inv(X) \leqslant n^2/c$.*

The algorithm that achieves this bound is essentially the Block Routine with block length $\ell = n/8$. when sorting in Steps 1 and 2, Mergesort is applied. In Step 5 we deviate from earlier schemes by taking advantage of the fact that the elements removed in Step 4 are organized in $(n/\ell) - 1 = 7$ runs. That is, the d_i elements removed to make X_iX_{i+1} a sorted sequence form a run if we take the $d_i/2$ elements removed from the front of X_{i+1} and put the $d_i/2$ elements removed from the end of X_i immediately after. In Step 5, these runs are simply merged pairwise until there is just one run left.

Let $C_i(n)$ denote the number of comparisons performed in Step i. For simplicity, assume that n is a power of 2. Recall that Mergesort sorts a sequence of length n in $M(n) = n \log n - n + 1$ comparisons [21]. Then

$$C_1(n) = 8 \cdot M\left(\frac{n}{8}\right) = n \log n - 4n + O(1)$$

and $C_2(n) = M(8) = O(1)$. Permuting the blocks in Step 3 requires no comparisons and hence $C_3(n) = O(1)$. As outlined in the proof of Lemma 2, Step 4 is carried out by concurrent binary searches which take

$$C_4(n) = O\left(\left(\frac{n}{\ell}\right) \log \ell\right) = O(\log n)$$

comparisons. Performing the actual removals costs no comparisons. Merging the removed elements organized in 7 runs takes

$$C_5(n) = R_X\left(\frac{n}{8}\right) \lceil \log 7 \rceil = 3 \cdot R_X\left(\frac{n}{8}\right)$$

comparisons. Finally, merging two sequences takes $C_6(n) = n-1$ comparisons. Adding up gives that the number of comparisons spent is

$$C(n) = n\log n - 3n + 3 \cdot R_X\left(\frac{n}{8}\right) + O(\log n). \tag{9}$$

For the information-theoretic average-case lower bound for comparison-based sorting algorithms, $\log(n!)$, Stirling's approximation gives

$$\log(n!) > n\log n - 1.45n + O(\log n). \tag{10}$$

From Eqs. (9) and (10) we conclude that, for sufficiently large n, $C(n) < \log(n!)$ if $R_X(n/8) \leqslant n/2$. If this does not hold, Lemma 4 gives

$$\frac{n}{2} < R_X\left(\frac{n}{8}\right) \leqslant \sqrt{\frac{12n \cdot Inv(X)}{n/8}},$$

implying that $Inv(X) > n^2/(3 \cdot 2^7)$.

Hence, if n is a sufficiently large power of 2, the constant c in the statement of Theorem 17 is at most $3 \cdot 2^7$. Otherwise, it becomes somewhat larger. This completes the proof of Theorem 17.

Note that the outlined algorithm can be viewed as a variant of Mergesort in the following sense:

1. Run Mergesort until there are just 8 runs.
2. Permute the runs.
3. Trim the runs such that their concatenation forms a single run.
4. Complete Mergesort on the trimmed off runs.
5. Merge.

In ordinary Mergesort all elements take part in $\log n$ merges, while in this algorithm all elements take part in $\log n - \log 8 + 1 = \log n - 2$ merges and $R_X(n/8)$ elements participate in $\lceil \log 7 \rceil = 3$ additional merges. This algorithm is tailored to maximize the number of inversions that can be exploited efficiently. Apparently it cannot gain more than a linear number of comparisons compared to Mergesort, independently of how few inversions there happens to be. To overcome this weakness one could, again, estimate $Inv(X)$ by taking a sample of sublinear size and use this as a guide for deciding when to interrupt Mergesort and start Step 2.

7. Parallel algorithms

Worst-case optimal parallel sorting algorithms have been known for around a decade; see e.g. [1, 6]. *Adaptive* parallel sorting has hardly received any attention at all yet. To our knowledge, the only existing results are: a *Max*-optimal algorithm by Altman and Chlebus [2], where *Max* tells the maximum distance that an element is from its correct position; a *Runs*-optimal algorithm by Chen and Carlsson [4], where *Runs* tells the number of ascending runs; an *Osc*-optimal algorithm by Chen and Levcopoulos

[5], where *Osc* tells the amount of 'oscillation' [19]; and a *Rem*-optimal algorithm by Levcopoulos and Petersson [17].

The model of computation used in all the above-listed adaptive algorithms is the PRAM (parallel random access machine) [14]. A PRAM is a synchronous parallel machine with processors numbered $1, 2, \ldots$, and with a global memory accessible to all processors. There are many variations of the PRAM based on the handling of simultaneous access to the same memory cell in global memory. We will assume the least powerful variant, namely the EREW (exclusive read exclusive write) PRAM, in which neither simultaneous reads nor simultaneous writes may occur.

A parallel algorithm that runs in $t(n)$ time using $p(n)$ processors is said to be *cost optimal* if $t(n) \cdot p(n)$ is proportional to the time complexity of the fastest known sequential algorithm for the problem considered. Hence, in the case of sorting with respect to inversions, a parallel algorithm is optimal if $t(n)p(n) = O(n \log(Inv(X)/n))$.

In this section we describe how to implement the Block Routine to obtain an $O(\log n)$-time EREW PRAM algorithm which is cost-optimal with respect to the number of inversions. More precisely,

Theorem 18. *Given p EREW PRAM processors, there is an algorithm that sorts any sequence X of length n in time*

$$O\left(\frac{n \log(Inv(X)/n)}{p} + \log n\right),$$

which is optimal with respect to Inv.

We prove the theorem by first giving two similar yet different implementations of the Block routine depending on how many processors are available. Then, in Section 7.3 we show how to combine the two algorithms.

Both implementations make frequent use of Cole's Mergesort, by which n elements can be sorted in $O(\log n)$ time using a linear number of EREW PRAM processors. Whenever we write 'sort' without specifying how, it should be understood that it is Cole's algorithm that is applied. We will further rely on that two sorted sequences of altoghether n elements can be merged in $O(\log n)$ time using $O(n/\log n)$ EREW PRAM processors [3].

7.1. There are many processors

Suppose that we are given $n \log I / \log n$ processors, where $I \geqslant (\log n)^{1/4}$. We show that then we can sort any sequence X of length n with $Inv(X) \leqslant nI$ in $O(\log n)$ time. As we have discussed before, this kind of a priori knowledge of the input is not realistic. However, the algorithm presented in this subsection is merely a *tool*; in Section 7.3 we describe how it will be used in the design of an algorithm that does not assume a priori knowledge of the number of inversions. Below, we thus assume that the above conditions hold.

We show that by choosing

$$\ell = \min\{n, \max\{(\log n)^3, I \cdot (\log n)^2\}\},$$

each step of the Block Routine can be implemented to run in $O(\log n)$ time. If $\ell = n$ only Step 1 is executed, which takes $O((\log n)^2/\log I)$ time, which is $O(\log n)$, since it would imply $I \geqslant n/(\log n)^2$. Hence, below we can assume that $\ell = \max\{(\log n)^3, I \cdot (\log n)^2\}$.

Step 1: *Sort blocks.* Distribute the processors evenly among the blocks and sort each block using Cole's Mergesort. Each block X_i is sorted by $\Theta(\ell \log I/\log n)$ processors, which takes time

$$O\left(\frac{\ell \log \ell}{\ell \log I/\log n}\right) = O\left(\frac{\log \ell \log n}{\log I}\right).$$

But this is bounded by $O(\log n)$ because

$$\begin{aligned}\log \ell &= \log(\max\{(\log n)^3, I \cdot (\log n)^2\}) \\ &= \Theta(\max\{\log, \log n, \log I\}) \\ &= \Theta(\log I),\end{aligned}$$

the last equality following from the assumption that $I \geqslant (\log n)^{1/4}$.

Step 2: *Sort medians.* Since the number of medians does not exceed the number of processors they can be sorted in time $O(\log n)$.

Step 3: *Permute blocks.* This step can trivially be done in $O(\log n)$, since we have less than $\log n$ elements per processor.

Step 4: *Trim blocks.* Assign one processor to each consecutive pair of blocks, X_i and X_{i+1}. Each processor then proceeds as described in the proof of Lemma 2, which takes $O(\log \ell) = O(\log n)$ time. Removing elements and compacting the sequence amounts to a prefix sums computation, which takes $O(\log n)$ time using $O(n/\log n)$ processors [14].

Step 5: *Sort removed elements.* The analysis in Section 5.1 shows that independently of whether $\ell = (\log n)^3$ or $\ell = I \cdot (\log n)^2$, we have $R_X(\ell) = O(n/\log n)$. Hence, in either case we have fewer elements to sort than the number of available processors, and so, the sort can be carried out in $O(\log n)$ time.

Step 6: *Merge.* Use the algorithm of Bilardi and Nicolau [3] to merge the two sorted sequences in $O(\log n)$ time using $n/\log n$ processors.

We have thus proved

Lemma 19. *Given $n \log I/\log n$ EREW PRAM processors, where $I \geqslant (\log n)^{1/4}$, there is an algorithm that sorts any sequence X of length n with $Inv(X) \leqslant nI$ in $O(\log n)$ time.*

7.2. There are few processors

As we already experienced in the sequential model of computation, when the input sequence has too few inversions we have to modify the Block Routine slightly to achieve *Inv*-optimality. Again, let us assume that we are given $n \log I / \log n$ processors, where, in this case, $I < (\log n)^{1/4}$, and where nI is an upper bound on the number of inversions in X. We show that by choosing block length $\ell = \sqrt{\log n}$, the Block Routine can be implemented to run in $O(\log n)$ time.

Step 1: *Sort blocks.* Put $N = n \log I$. Then we can state the problem as that of processing $n/\sqrt{\log n} \leqslant N$ tasks, each requiring $O(\sqrt{\log n} \log\log n) = O(\log N)$ time, and the total number of operations required to carry out all the tasks is $O(N)$ (if we sort each block sequentially using an *Inv*-optimal sorting algorithm, such as Splitsort). This is an instance of the *duration-unknown task scheduling problem*, which can be solved in $O(\log N)$ time using $O(N/\log N)$ processors on an EREW PRAM, due to a processor scheduling technique of Cole and Vishkin [7]. The blocks can thus be sorted in time $O(\log N) = O(\log n)$, and since

$$\frac{n \log I}{\log n} = \Omega\left(\frac{N}{\log n}\right) = \Omega\left(\frac{N}{\log N}\right),$$

the number of available processors is sufficient.

Step 2: *Sort medians.* Denote the sequence of medians by X', and put $N = |X'| = n/\sqrt{\log n}$. It is important that X' is a subsequence of X, that is, the medians must appear in the same order in X' as in X. We show that we have enough processors to apply the algorithm of Lemma 19. In order to express the number of inversions among the medians, $Inv(X')$, in terms of N, we first note that $n = \Theta(N\sqrt{\log N})$ and $\log n = \Theta(\log N)$. Hence,

$$\begin{aligned} Inv(X') &\leqslant Inv(X) \leqslant nI \\ &= \Theta(N\sqrt{\log N} \cdot I) \\ &= O(N\sqrt{\log N} \cdot (\log n)^{1/4}) \\ &= O(N(\log N)^{3/4}). \end{aligned}$$

The number of available processors is at least

$$\frac{n}{\log n} = \frac{N}{\sqrt{\log n}} = \Theta\left(\frac{N\sqrt{\log N}}{\log N}\right) = \Omega\left(\frac{N \log\log N}{\log N}\right).$$

Hence, we have $\Omega(N \log I'/\log N)$ processors, where $I' \geqslant \log N$. As $I' \geqslant (\log N)^{1/4}$ and $Inv(X') \leqslant nI'$, the algorithm of Lemma 19 can indeed be applied to sort X' in $O(\log N) = O(\log n)$ time.

Step 3: *Permute blocks.* This step can, as was the case for the algorithm in Section 7.1, be done in $O(\log n)$ time since we have less than $\log n$ elements per processor.

Step 4: *Trim blocks.* The same argument as in Step 4 in Section 7.1 shows that $O(\log n)$ time suffices.

Step 5: *Sort removed elements.* Denote the sequence of removed elements by X', and put $N = |X'|$. As in Step 2 it is important that X' is a subsequence of the concatenation of the blocks $X_1X_2 \cdots X_{n/\ell}$. Now, since $Inv(X) < n(\log n)^{1/4}$, Lemma 4 implies

$$N = R_X(\sqrt{\log n}) < \sqrt{\frac{12n^2(\log n)^{1/4}}{\sqrt{\log n}}} = O\left(\frac{n}{(\log n)^{1/8}}\right).$$

If $N \leqslant n/\log n$ Cole's Mergesort will complete the sort in $O(\log n)$ time. Otherwise, we show that we can apply the algorithm of Lemma 19. So assume that $N > n/\log n$. Then $\log n = \Theta(\log N)$ and $n = O(N \log N)$. The same argument as in Step 2 now gives

$$\begin{aligned} Inv(X') &= O(N \log N \cdot I) \\ &= O(N \log N \cdot (\log n)^{1/4}) \\ &= O(N(\log N)^{5/4}). \end{aligned}$$

The number of processors available is at least

$$\frac{n}{\log n} = \Omega\left(\frac{N(\log n)^{1/8}}{\log n}\right) = \Omega\left(\frac{N(\log N)^{1/8}}{\log N}\right) = \Omega\left(\frac{N \log(\log N)^2}{\log N}\right).$$

Hence, we have $\Omega(N \log I'/\log N)$ processors, where $I' \geqslant (\log N)^2$. Since $I' \geqslant (\log N)^{1/4}$ and $Inv(X') \leqslant NI'$, X' can be sorted in $O(\log N) = O(\log n)$ time by the algorithm of Lemma 19.

Step 6: *Merge.* Again, as in Section 7.1, this step can be done in $O(\log n)$ time.

We summarize the above analysis in the following lemma.

Lemma 20. *Given $n \log I/\log n$ EREW PRAM processors, where $I < (\log n)^{1/4}$, there is an algorithm that sorts any sequence X of length n with $Inv(X) \leqslant nI$ in $O(\log n)$ time.*

7.3. Processor scheduling

In the previous two subsections we have demonstrated that if we know the number of inversions in a sequence to be sorted, we can allocate an appropriate number of processors, that is, $n \log(Inv(X)/n)/\log n$, and sort optimally in $O(\log n)$ time. A more realistic situation is that we are alloted p processors.

Our goal must be to sort in time

$$O\left(\frac{n \log(Inv(X)/n)}{p} + \log n\right).$$

Readers familiar with parallel computation might at first think that this is easily accomplished by slowing down the algorithms of Lemmas 19 and 20. If we knew $Inv(X)$ in advance this would be true: We first determine $I = Inv(X)/n$ to decide which algorithm to apply. We then compute

$$s = \frac{n \log I / \log n}{p},$$

and let each (real) processor simulate s virtual processors.

If we do not know $Inv(X)$ things are not as easy. Note that, the naive strategy of computing I such that $p = n \log I / \log n$, and running the appropriate algorithm (depending on the relation between I and $(\log n)^{1/4}$) does not necessarily work the way one might expect. That is, it might result in a slowdown much bigger than s. The reason is that the analyses leading to Lemmas 19 and 20 make use of that I bounds $Inv(X)$ from above. If this is not the case, we have no guarantee whatsoever on the running times of the underlying algorithms; at least not in terms of $Inv(X)$.

The main idea in our scheme for scheduling the p processors is that, based on the number of processors we are given, we *guess* the number of inversions in X. To evaluate our guess we start the appropriate algorithm of Lemma 19 or Lemma 20, and wait to see if it stops within a certain amount of time. If it does not stop, we abort it, increase our guess, and repeat the procedure (with the appropriate slowdown), and so on.

In the following, let c be a constant such that any of the algorithms of Lemma 19 and lemma 20 would sort X in at most $c \cdot \log n$ steps given $n \log(Inv(X)/n)/\log n$ processors.

First compute I such that $p = n \log I / \log n$, and guess that $Inv(X) \leqslant nI$. Depending on the relation between I and $(\log n)^{1/4}$, start the algorithm of Lemma 19 or that of Lemma 20. If $Inv(X) \leqslant nI$, the algorithm stops within $c \cdot \log n$ steps, in which case we are done.

Otherwise, the guess was too small, that is, $Inv(X) > nI$. Then we abort the algorithm, revise the guess to $Inv(X) \leqslant nI^2$, and restart it; this time letting every processor simulate two virtual processors. If it does not stop within $2c \cdot \log n$ steps ($c \cdot \log n$ steps per virtual processor) we abort it again, and so on. In the ith iteration we guess that $Inv(X) \leqslant nI^{2^i}$, let each processor simulate 2^i virtual processors, and allow $2^i c \cdot \log n$ steps before aborting the computation. If, initially, $I < (\log n)^{1/4}$ we proceed in the same way, but apply the algorithm of Lemma 20 instead; however, as soon as $I^{2^i} \geqslant (\log n)^{1/4}$ we switch to the algorithm of Lemma 19.

Eventually, the algorithm will complete the sort without being interrupted. Suppose this happens in the kth iteration. As the number of steps is doubled in each iteration the total time spent is proportional to that spent in the last iteration, that is, $O(2^k \log n)$. It remains to bound k from above. Since we did not finish in the $(k-1)$th iteration, $Inv(X) > nI^{2^{k-1}}$. But this implies that $k = O(\log(\log(Inv(X)/n)/\log I))$. Hence, the

total time spent is bounded by

$$O\left(2^{\log(\log(Inv(X)/n)/\log I)}\log n\right) = O\left(\frac{\log(Inv(X)/n)}{\log I}\log n\right)$$
$$= O\left(\frac{n\log(Inv(X)/n)}{p} + \log n\right),$$

with the last equality following by observing that $\Omega(\log n)$ time is always spent, and by replacing $\log I$ by $p\log n/n$. This concludes the proof of Theorem 18.

8. Conclusions

We have introduced a simple generic algorithm that adapts to the number of inversions; the Block Routine. We first divide the input sequence into blocks of equal length and sort each block separately. The blocks are permuted such that their medians are sorted. We then chop off portions in the beginning and in the ends of the blocks such that their concatenation forms a sorted sequence. The chopped off portions are sorted separately and finally merged with the concatenated sequence. The intuition behind the Block Routine is that if the length of the blocks is chosen appropriately, the number of elements chopped off is quite small.

Using the Block Routine, we constructed the sorting algorithm Blocksort, which is optimal with respect to the number of inversions, as long as this number is fairly large. We then turned to an in-place implementation of Blocksort, which was proven optimal with respect to *Inv* and *Rem*.

Since it relies on repeated median finding, Blocksort may not be the method of choice in practical applications. We therefore showed that median finding can be avoided by estimating the number of inversions by means of a random sample. The resulting algorithm is simple, and efficient with high probability.

We then showed how to implement the generic algorithm in order to save comparisons when there are many inversions. Finally, we described how the Block Routine can be implemented to achieve optimal speedup on the EREW PRAM.

Acknowledgements

We would like to thank the anonymous referee for insightful comments which greatly helped to improve the readability of this paper.

References

[1] M. Ajtai, J. Komlós and E. Szemeredi, Sorting in $c\log n$ parallel steps, *Combinatorica* **3** (1983) 1–19.
[2] T. Altman and B.S. Chlebus, Sorting roughly sorted sequences in parallel, *Inform. Process. Lett.* **33** (1990) 297–300.

[3] G. Bilardi and A. Nicolau, adaptive bitonic sorting: An optimal parallel algorithm for shared memory models, *SIAM J. Comput.* **18** (1989) 216–228.
[4] J. Chen and S. Carlsson, An Optimal parallel adaptive sorting algorithm, *Inform. Process. Lett.* **39** (1991) 195–200.
[5] J. Chen and C. Levcopoulos, Improved parallel sorting of presorted files, in: *Proc. Joint Conf. on Vector and Parallel Processing*, Lecture Notes in Computer Science, Vol. 634 (Springer, Berlin, 1992) 539–544.
[6] R. Cole, Parallel merge sort, *SIAM J. Comput.* **17** (1988) 770–785.
[7] R. Cole and U. Vishkin, Approximate parallel scheduling. Part 1: The basic technique with applications to optimal parallel list ranking in logarithmic time, *SIAM J. Comput.* **17** (1988) 128–142.
[8] E.W. Dijkstra, Smoothsort, an alternative to sorting in situ, *Sci. Comput. Programming* **1** (1982) 223–233.
[9] R.W. Floyd AND R.L. Rivest, Expected time bounds for selection, *Comm. ACM* **18** (1975) 165–172.
[10] L.J. Guibas, E.M. McCreight, M.F. Plass and J.R. Roberts, A new representation of linear lists, in: *Proc. 9th Ann. ACM Symp. on Theory of Computing* (1977) 49–60.
[11] T. Hagerup and C. Rüb, A guided tour of Chernoff bounds, *Inform. Process. Lett.* **33** (1989) 305–308.
[12] S. HERTEL, Smoothsort's behaviour on presorted sequences, *Inform. Process. Lett.* **16** (1983) 165–170.
[13] B.-C. Huang and M.A. Langston, Practical in-place merging, *Comm. ACM*, **31** (1988) 348–352.
[14] J. JÁJÁ, *An Introduction to Parallel Algorithms* (Addison-Wesley, Reading, MA, 1992).
[15] D.E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching* (Addition-Wesley, Reading, MA, 1973).
[16] T.W. Lai and D. Wood, Implicit selection, in: *Proc. 1st Scandinavian Workshop on Algorithm Theory*, Lecture Notes in Computer Science, Vol. 318 (Springer, Berlin, 1988) 14–23.
[17] C. Levcopoulos and O. Petersson, A note on adaptive parallel sorting, *Inform. Process. Lett.* **33** (1989) 187–191.
[18] C. Levcopoulos and O. Petersson, Splitsort – an adaptive sorting algorithm, *Inform. Process. Lett.* **39** (1991) 205–211.
[19] C. Levcopoulos and O. Petersson, Adaptive Heapsort, *J. Algorithms* **14** (1993) 395–413.
[20] H. Mannila, Measures of presortedness and optimal sorting algorithms, *IEEE Trans. Comput.* **C-34** (1985) 318–325.
[21] K. Mehlhorn, *Data Structures and Algorithms, Vol. 1: Sorting and Searching* (Springer, Berlin, 1984).
[22] J. I. Munro, An implicit data structure supporting insertion, deletion, and search in $O(\log^2 n)$ time, *J. Comput. System. Sci.* **33** (1986) 66–74.
[23] O. Petersson and A.M. Moffat, A framework for adaptive sorting, *Discrete Applied Math.*, to appear. Prel. version in *Proc. 3rd Scandinavian Workshop on Algorithm Theory*, Lecture Notes in Computer Science, Vol. 621 (Springer, Berlin, 1992) 422–433.